Especificación de Protocolo

RFC-CGSP-2

CyberGame Socket Protocol (CGSP)

Versión 2.0

Estado: Propuesto
Fecha: Abril 2026
Materia: Internet: Arquitectura y Protocolos — 2026-1
Autores: Equipo Entrega_1_Telematica

Abstract. Este documento especifica el *CyberGame Socket Protocol (CGSP)*, un protocolo de la capa de aplicación diseñado para soportar la comunicación en tiempo real entre clientes y el servidor central de un simulador interactivo multijugador de ciberseguridad. El protocolo define el formato de mensajes, el vocabulario de comandos, las reglas de procedimiento y el manejo de errores para una partida en la que jugadores con roles de "Atacante" o "Defensor" interactúan sobre un plano virtual compartido.

Universidad EAFIT — Departamento de Ingeniería de Sistemas

Índice

1. Visión General del Protocolo **3**
1.1. Propósito . . . 3
1.2. Modelo de Funcionamiento . . . 3
1.3. Capa de la Arquitectura TCP/IP . . . 3
1.4. Resolución de Nombres . . . 4

2. Especificación del Servicio **4**
2.1. Primitivas del Servicio . . . 4
2.1.1. Estado: CONECTADO (antes de autenticarse) . . . 4
2.1.2. Estado: AUTENTICADO . . . 5
2.1.3. Estado: EN PARTIDA . . . 5
2.1.4. Mensajes Asíncronos del Servidor . . . 5

3. Formato de Mensajes **6**
3.1. Codificación General . . . 6
3.2. Mensajes del Cliente hacia el Servidor . . . 6
3.2.1. AUTH — Autenticación . . . 6
3.2.2. LIST_ROOMS — Listar Salas . . . 6
3.2.3. CREATE_ROOM — Crear Sala . . . 7
3.2.4. JOIN — Unirse a Sala . . . 7
3.2.5. START — Iniciar Partida . . . 7
3.2.6. MOVE — Mover Jugador . . . 7
3.2.7. SCAN — Escanear Área . . . 8
3.2.8. ATTACK — Atacar Recurso . . . 8
3.2.9. DEFEND — Defender Recurso . . . 8
3.2.10. STATUS — Consultar Estado . . . 8
3.2.11. QUIT — Cerrar Sesión . . . 8
3.3. Mensajes del Servidor hacia el Cliente . . . 9
3.3.1. OK — Operación Exitosa . . . 9
3.3.2. ROLE — Rol Asignado . . . 9
3.3.3. ERR — Error . . . 9
3.3.4. ROOM_CREATED — Sala Creada . . . 10
3.3.5. ROOM_LIST — Lista de Salas . . . 10
3.3.6. Eventos del Servidor . . . 10

4. Protocolo del Servicio de Identidad **11**
4.1. Modelo de Comunicación . . . 11
4.2. Formato de Mensajes . . . 11
4.3. Ejemplo de Intercambio . . . 11
4.4. Configuración . . . 12

5. Reglas de Procedimiento (Flujo de Operación) **12**
5.1. Máquina de Estados del Cliente . . . 13

5.2. Ciclo de Vida de una Sala 13
5.3. Condiciones de Victoria 13
5.4. Flujo de Ataque y Defensa 14
5.5. Flujo de Exploración del Atacante 14
5.6. Manejo de Errores y Excepciones 15

6. Ejemplos de Implementación **15**
6.1. Sesión Completa de un Atacante 15
6.2. Sesión Completa de un Defensor 16
6.3. Escenario: Ataque no Defendido (victoria atacante) 17
6.4. Diagrama de Secuencia de una Partida Completa 17
6.5. Prueba Manual con Telnet 18

7. Parámetros del Servidor **19**

8. Parámetros del Mapa de Juego **19**

9. Consideraciones de Seguridad **19**

1. Visión General del Protocolo

1.1. Propósito

CGSP permite que múltiples jugadores interactúen simultáneamente con un servidor de juego central. A través de este protocolo, los jugadores pueden:

- Autenticarse en el sistema mediante un servicio de identidad externo.
- Listar y unirse a salas de juego activas.
- Moverse por un plano virtual bidimensional.
- Ejecutar acciones específicas de su rol (atacar o defender recursos críticos).
- Recibir notificaciones de eventos del sistema en tiempo real.

1.2. Modelo de Funcionamiento

CGSP adopta el modelo **cliente-servidor** con comunicación **bidireccional persistente**. El siguiente diagrama ilustra la arquitectura del sistema:

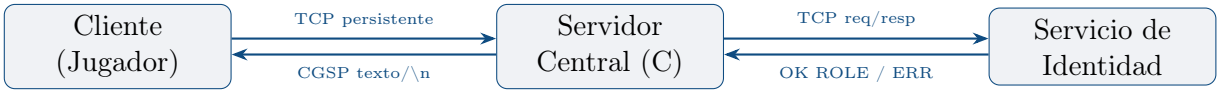

Figura 1: Arquitectura del sistema CGSP

El servidor acepta conexiones TCP entrantes y crea un hilo POSIX (`pthread`) dedicado por cada cliente. La conexión se mantiene activa durante toda la sesión del jugador. El servidor también actúa como cliente frente al Servicio de Identidad para autenticar usuarios.

1.3. Capa de la Arquitectura TCP/IP

CGSP opera en la **capa de aplicación** del modelo TCP/IP:

Capa	Protocolo
Aplicación	**CGSP v2.0** (este documento)
Transporte	TCP (SOCK_STREAM)
Red	IPv4
Enlace	Ethernet / Wi-Fi

Cuadro 1: Pila de protocolos de CGSP

> **Justificación de TCP sobre UDP.** El juego requiere confiabilidad absoluta en la entrega de mensajes. Un paquete de `ATTACK` o `DEFEND` que se pierda comprometería la consistencia del estado del juego entre servidor y clientes. TCP garantiza entrega ordenada, detección de pérdidas y retransmisión automática. UDP sería adecuado solo para actualizaciones de posición de alta frecuencia donde una pérdida es aceptable — lo cual no aplica a este diseño.

1.4. Resolución de Nombres

CGSP prohíbe el uso de direcciones IP codificadas (*hardcoded*) en el software. Toda referencia a servicios externos (servidor de juego, servicio de identidad) se resuelve mediante DNS usando la función `getaddrinfo()` de POSIX. Si la resolución falla, el servicio maneja la excepción sin finalizar su ejecución, respondiendo al cliente con `ERR 503`.

2. Especificación del Servicio

2.1. Primitivas del Servicio

Las primitivas de CGSP se organizan según el estado del cliente en el servidor.

2.1.1. Estado: CONECTADO (antes de autenticarse)

Primitiva	Dirección	Descripción
`AUTH`	Cliente → Servidor	Autenticar un usuario contra el servicio de identidad
`QUIT`	Cliente → Servidor	Cerrar la conexión ordenadamente

Cuadro 2: Primitivas disponibles en estado CONECTADO

2.1.2. Estado: AUTENTICADO

Primitiva	Dirección	Descripción
`LIST_ROOMS`	Cliente → Servidor	Obtener lista de salas activas
`CREATE_ROOM`	Cliente → Servidor	Crear una nueva sala de juego
`JOIN`	Cliente → Servidor	Unirse a una sala existente
`STATUS`	Cliente → Servidor	Consultar estado actual del jugador
`QUIT`	Cliente → Servidor	Cerrar la conexión

Cuadro 3: Primitivas disponibles en estado AUTENTICADO

2.1.3. Estado: EN PARTIDA

Primitiva	Dirección	Rol	Descripción
`MOVE`	C → S	Ambos	Mover al jugador en el plano
`SCAN`	C → S	ATTACKER	Detectar recursos críticos cercanos
`ATTACK`	C → S	ATTACKER	Lanzar un ataque sobre un recurso
`DEFEND`	C → S	DEFENDER	Mitigar un ataque activo
`START`	C → S	Ambos	Intentar iniciar la partida
`STATUS`	C → S	Ambos	Consultar estado del jugador/sala
`QUIT`	C → S	Ambos	Abandonar la partida

Cuadro 4: Primitivas disponibles EN PARTIDA

2.1.4. Mensajes Asíncronos del Servidor

Evento	Receptores	Descripción
`EVENT PLAYER_JOINED`	Sala completa	Un nuevo jugador se unió a la sala
`EVENT PLAYER_LEFT`	Sala completa	Un jugador abandonó la sala
`EVENT GAME_STARTED`	Sala completa	La partida comenzó
`EVENT RESOURCE_INFO`	Solo defensores	Coordenadas de recursos críticos (al iniciar)
`EVENT RESOURCE_FOUND`	Atacantes de la sala	Recurso detectado por SCAN
`EVENT ATTACK`	Sala completa	Notificación de ataque activo
`EVENT DEFENDED`	Sala completa	Recurso fue defendido exitosamente
`EVENT ATTACK_TIMEOUT`	Sala completa	Recurso comprometido por expiración del timer
`EVENT GAME_OVER`	Sala completa	Fin de partida con resultado

Cuadro 5: Mensajes asíncronos del servidor (broadcast y unicast)

3. Formato de Mensajes

3.1. Codificación General

Todo mensaje CGSP debe cumplir las siguientes reglas de codificación:

- **Codificación de caracteres:** UTF-8.
- **Delimitador de mensaje:** `\n` (LF, `0x0A`). Un mensaje por línea.
- **Separador de campos:** espacio (`0x20`).
- **Longitud máxima de mensaje:** 2 048 bytes.
- **Sensibilidad a mayúsculas en comandos:** NO (el servidor normaliza a mayúsculas).
- **Timeout de inactividad:** 600 segundos por defecto (configurable por `CGSP_IDLE_TIMEOUT`). El servidor enviará `ERR 408 Timeout: conexion cerrada por inactividad` y cerrará la conexión si el cliente no envía ningún mensaje en ese intervalo.

Estructura general:

```
COMANDO [PARAM1] [PARAM2] ... [PARAMn]\n
```

3.2. Mensajes del Cliente hacia el Servidor

3.2.1. AUTH — Autenticación

```
AUTH <usuario> <password>\n
```

Campo	Tipo	Máx.	Descripción
`usuario`	string	63 chars	Nombre de usuario registrado
`password`	string	63 chars	Contraseña del usuario

Ejemplo: `AUTH carlos hack2026`

3.2.2. LIST_ROOMS — Listar Salas

```
LIST_ROOMS\n
```

Sin parámetros. Solicita la lista de salas de juego activas.

> **Información extendida de sala.** La respuesta `ROOM_LIST` incluye ahora el desglose de roles por sala (atacantes/defensores actuales), lo que permite al cliente tomar decisiones de balanceo antes de unirse. Ver sección 3.3.5.

3.2.3. CREATE_ROOM — Crear Sala

```
CREATE_ROOM\n
```

Sin parámetros. Crea una nueva sala de juego y coloca los recursos críticos en posiciones aleatorias del mapa.

3.2.4. JOIN — Unirse a Sala

```
JOIN <room_id>\n
```

Campo	Tipo	Descripción
`room_id`	entero positivo	Identificador numérico de la sala

Ejemplo: `JOIN 42`

3.2.5. START — Iniciar Partida

```
START\n
```

Solicita el inicio de la partida. Solo tiene efecto si hay al menos un atacante y un defensor en la sala.

3.2.6. MOVE — Mover Jugador

```
MOVE <dx> <dy>\n
```

Campo	Tipo	Rango	Descripción
`dx`	entero	$[-100, 100]$	Desplazamiento en el eje X
`dy`	entero	$[-100, 100]$	Desplazamiento en el eje Y

El servidor clampea la posición resultante dentro del mapa $[0, 99] \times [0, 99]$.
Ejemplo: `MOVE 5 -3`

3.2.7. SCAN — Escanear Área

```
SCAN\n
```

Solo para jugadores con rol `ATTACKER`. Detecta recursos críticos en un radio de **12 unidades** alrededor de la posición actual del atacante. La detección se calcula con distancia euclidiana:

$$d = \sqrt{(x_{jugador} - x_{recurso})^2 + (y_{jugador} - y_{recurso})^2} \leq 12$$

3.2.8. ATTACK — Atacar Recurso

```
ATTACK <resource_id>\n
```

Campo	Tipo	Descripción
`resource_id`	entero (0 ó 1)	Identificador del recurso crítico a atacar

Ejemplo: `ATTACK 0`

3.2.9. DEFEND — Defender Recurso

```
DEFEND <resource_id>\n
```

Campo	Tipo	Descripción
`resource_id`	entero (0 ó 1)	Identificador del recurso a defender

Ejemplo: `DEFEND 0`

3.2.10. STATUS — Consultar Estado

```
STATUS\n
```

3.2.11. QUIT — Cerrar Sesión

```
QUIT\n
```

3.3. Mensajes del Servidor hacia el Cliente

3.3.1. OK — Operación Exitosa

```
OK <mensaje>\n
```

Ejemplos:

```
OK Bienvenido carlos
OK Posicion: 15 42
OK Ataque lanzado
OK Recurso defendido
OK Hasta luego
```

3.3.2. ROLE — Rol Asignado

```
ROLE <ATTACKER|DEFENDER>\n
```

Enviado inmediatamente después de un `AUTH` exitoso.

3.3.3. ERR — Error

```
ERR <codigo> <descripcion>\n
```

Código	Nombre	Descripción
400	BAD_REQUEST	Formato de mensaje inválido o parámetros faltantes
401	UNAUTHORIZED	Autenticación fallida o acción sin autenticar previamente
403	FORBIDDEN	El rol del jugador no permite esta acción
404	NOT_FOUND	Sala o recurso no encontrado
408	TIMEOUT	Conexión cerrada por inactividad
409	CONFLICT	Conflicto de estado (sala llena, partida iniciada, recurso ya atacado)
500	INTERNAL_ERROR	Error interno del servidor
503	SERVICE_UNAVAILABLE	Servicio de identidad no disponible

Cuadro 6: Tabla de códigos de error de CGSP

Ejemplos:

```
ERR 401 Credenciales incorrectas
ERR 403 Solo los atacantes pueden usar ATTACK
ERR 404 Sala no encontrada
ERR 408 Timeout: conexion cerrada por inactividad
ERR 409 Recurso ya bajo ataque
ERR 503 Servicio de identidad no disponible, intenta luego
```

3.3.4. ROOM_CREATED — Sala Creada

```
ROOM_CREATED <room_id>\n
```

Ejemplo: `ROOM_CREATED 7`

3.3.5. ROOM_LIST — Lista de Salas

```
ROOM_LIST <n>\n
ROOM <id> <WAITING|RUNNING|FINISHED> <atacantes>/<defensores
   >/<max>\n
...
```

El campo de ocupación ahora desglosa `<atacantes>/<defensores>/<max>`, permitiendo al cliente evaluar el balance de roles antes de unirse.

Ejemplo:

```
ROOM_LIST 2
ROOM 1 WAITING 1/0/10
ROOM 5 RUNNING 2/3/10
```

3.3.6. Eventos del Servidor

```
EVENT PLAYER_JOINED <usuario> <ATTACKER|DEFENDER>\n
EVENT PLAYER_LEFT <usuario>\n
EVENT GAME_STARTED\n
EVENT RESOURCE_INFO <id> <x> <y>\n
EVENT RESOURCE_FOUND <id> <x> <y>\n
EVENT ATTACK <resource_id> <atacante>\n
EVENT DEFENDED <resource_id> <defensor>\n
EVENT ATTACK_TIMEOUT <resource_id>\n
EVENT GAME_OVER <ATTACKER|DEFENDER>\n
```

> **Unicast vs. Broadcast.** Los eventos `RESOURCE_INFO` se envían únicamente a jugadores con rol defensor cuando inicia la partida. Los eventos `RESOURCE_FOUND` se envían al atacante que ejecutó el `SCAN` y también a los demás atacantes de la misma sala. Todos los demás eventos se difunden a todos los jugadores de la sala (*broadcast*).

4. Protocolo del Servicio de Identidad

El Servicio de Identidad es un componente externo al que el servidor CGSP consulta para verificar credenciales de usuario. Esta sección especifica el sub-protocolo de comunicación entre el servidor CGSP y dicho servicio.

4.1. Modelo de Comunicación

El servidor CGSP actúa como **cliente TCP** frente al Servicio de Identidad. La conexión es de tipo **petición–respuesta**: se establece una conexión por cada autenticación, se envía una consulta, se recibe la respuesta, y la conexión se cierra.

4.2. Formato de Mensajes

Los mensajes siguen la misma codificación que CGSP: texto UTF-8 delimitado por `\n`.

Petición del servidor CGSP al Servicio de Identidad:

```
AUTH_CHECK <usuario> <password>\n
```

Respuestas del Servicio de Identidad:

```
OK <ATTACKER|DEFENDER>\n   % Credenciales validas; retorna el
   rol asignado
ERR 401 <descripcion>\n    % Credenciales invalidas
ERR 503 <descripcion>\n    % Servicio temporalmente no
   disponible
```

> El rol del jugador (`ATTACKER` o `DEFENDER`) es devuelto directamente por el Servicio de Identidad. El servidor CGSP no asigna roles de forma independiente, garantizando así consistencia entre partidas.

4.3. Ejemplo de Intercambio

```
# Servidor CGSP -> Servicio de Identidad
AUTH_CHECK carlos hack2026
```

```
# Servicio de Identidad -> Servidor CGSP  (credenciales OK)
OK ATTACKER

# Servicio de Identidad -> Servidor CGSP  (credenciales
   incorrectas)
ERR 401 Usuario o contrasena invalidos

# Servicio de Identidad -> Servidor CGSP  (servicio caido)
ERR 503 Base de datos no disponible
```

4.4. Configuración

El host y puerto del Servicio de Identidad se configuran mediante variables de entorno (ver sección 6). El servidor de juego nunca almacena credenciales ni resultados en caché; cada `AUTH` del cliente genera una nueva consulta al Servicio de Identidad.

5. Reglas de Procedimiento (Flujo de Operación)

5.1. Máquina de Estados del Cliente

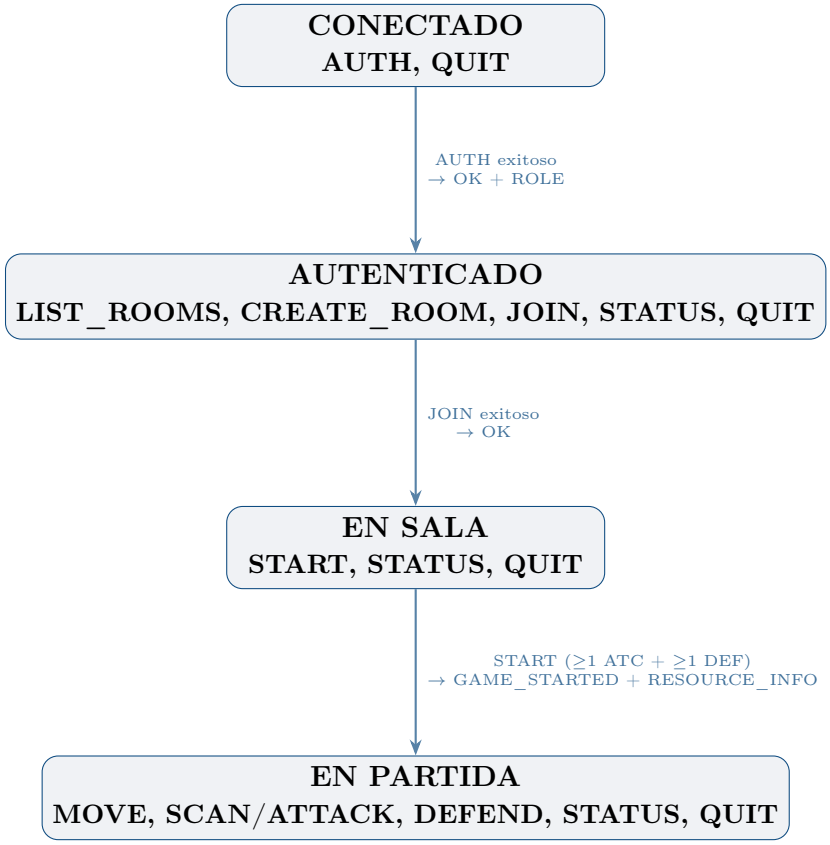

Figura 2: Máquina de estados del cliente CGSP

5.2. Ciclo de Vida de una Sala

Figura 3: Ciclo de vida de una sala de juego

5.3. Condiciones de Victoria

La partida finaliza cuando se cumple **cualquiera** de las siguientes condiciones, momento en que el servidor emite `EVENT GAME_OVER <ganador>` a todos los jugadores de la sala:

Ganador	Condición
ATTACKER	Al menos **un recurso crítico** no es defendido dentro del plazo de 30 s tras ser atacado. El evento EVENT ATTACK_TIMEOUT <resource_id> precede al EVENT GAME_OVER.
DEFENDER	**Todos los ataques** sobre ambos recursos son repelidos exitosamente y el equipo atacante ejecuta QUIT o la sala supera el tiempo máximo de partida (ver sección 7).

Cuadro 7: Condiciones de victoria de CGSP

> Si un atacante abandona la partida (QUIT) mientras hay un ataque en curso, dicho ataque se cancela automáticamente y se emite EVENT PLAYER_LEFT. Si todos los atacantes abandonan, el servidor declara victoria del equipo defensor.

5.4. Flujo de Ataque y Defensa

```
ATACANTE          SERVIDOR             DEFENSOR(ES)
   |                  |                    |
   |-- ATTACK 0 ----->|                    |
   |<- OK Ataque -----|                    |
   |                  |-- EVENT ATTACK 0 -->|
   |                  |     [timer: 30 s]   |
   |                  |                    |-- DEFEND 0 -->|
   |                  |<-------------------- DEFEND 0 ------|
   |                  |-- EVENT DEFENDED 0 maria ---------->|
   |<- EVENT DEFENDED |                    |

  (Si el timer expira sin DEFEND:)
   |                  |-- EVENT ATTACK_TIMEOUT 0 ---------->|
   |<- EVENT ATTACK_T.|                    |
   |                  |-- EVENT GAME_OVER ATTACKER -------->|
   |<- EVENT GAME_OVER|                    |
```

> **Timer de ataque.** Un recurso bajo ataque que no sea defendido dentro de **30 segundos** constituye victoria para el equipo atacante. El servidor emite EVENT ATTACK_TIMEOUT <resource_id> seguido de EVENT GAME_OVER ATTACKER a todos los jugadores de la sala.

5.5. Flujo de Exploración del Atacante

```
ATACANTE              SERVIDOR
   |                     |
   |-- MOVE 30 35 ------>|   pos = (30, 35)
```

```
   |<- OK Posicion 30 35 |
    |-- SCAN ------------>|   dist((30,35), recurso) <= 12 ?
   |<- EVENT RESOURCE_FOUND 0 32 37    (si detecta)
   |<- OK SCAN: no encontrado          (si no detecta)
   |-- ATTACK 0 -------->|
   |<- OK Ataque lanzado |
```

5.6. Manejo de Errores y Excepciones

Situación	Respuesta del Servidor
Línea vacía o sin `\n`	Ignorada silenciosamente
Comando desconocido	`ERR 400 Comando desconocido`
Parámetros faltantes	`ERR 400 Uso: CMD <params>`
Acción sin autenticar	`ERR 401 Debes autenticarte primero`
Rol incorrecto para acción	`ERR 403 Solo los [rol] pueden usar [CMD]`
Sala inexistente	`ERR 404 Sala no encontrada`
Recurso inexistente	`ERR 404 Recurso no encontrado`
Sala llena o partida iniciada	`ERR 409 <descripción del conflicto>`
Servicio de identidad caído	`ERR 503` — servidor **no** finaliza
Desconexión abrupta del cliente	`read()` retorna $\leq 0 \rightarrow$ cierre limpio del hilo
Inactividad > 600 s	`ERR 408 Timeout: conexion cerrada por inactividad` $\rightarrow$ cierre del hilo
AUTH con sesión ya activa	`ERR 403 Ya estas autenticado`
JOIN cuando ya está en sala	`ERR 403 Ya estas en una sala`

Cuadro 8: Tabla de manejo de excepciones del protocolo

6. Ejemplos de Implementación

6.1. Sesión Completa de un Atacante

```
# Conexion establecida
S->C: OK Bienvenido al servidor CyberGame (CGSP v2.0). Usa
   AUTH

# Autenticacion
C->S: AUTH carlos hack2026
```

```
S->C: OK Bienvenido carlos
S->C: ROLE ATTACKER

# Listar salas disponibles
C->S: LIST_ROOMS
S->C: ROOM_LIST 1
S->C: ROOM 3 WAITING 0/1/10   # 0 atacantes, 1 defensor, max
   10

# Unirse a sala
C->S: JOIN 3
S->C: OK Te uniste a la sala

# Iniciar partida
C->S: START
S->C: OK Partida iniciada
S->C: EVENT GAME_STARTED

# Explorar el mapa
C->S: MOVE 30 35
S->C: OK Posicion: 30 35

C->S: SCAN
S->C: EVENT RESOURCE_FOUND 0 32 37

# Lanzar ataque
C->S: ATTACK 0
S->C: OK Ataque lanzado

# Notificacion de defensa exitosa
S->C: EVENT DEFENDED 0 maria

C->S: QUIT
S->C: OK Hasta luego
```

6.2. Sesión Completa de un Defensor

```
C->S: AUTH maria seg2026
S->C: OK Bienvenido maria
S->C: ROLE DEFENDER

C->S: JOIN 3
S->C: OK Te uniste a la sala

C->S: START
S->C: EVENT GAME_STARTED
```

```
S->C: EVENT RESOURCE_INFO 0 32 37    # conoce los recursos
S->C: EVENT RESOURCE_INFO 1 75 82

C->S: MOVE 32 37
S->C: OK Posicion: 32 37

# Alerta de ataque
S->C: EVENT ATTACK 0 carlos

C->S: DEFEND 0
S->C: OK Recurso defendido

C->S: QUIT
S->C: OK Hasta luego
```

6.3. Escenario: Ataque no Defendido (victoria atacante)

```
# Timer de 30 s expira sin respuesta del defensor
S->C (ambos): EVENT ATTACK_TIMEOUT 0
S->C (ambos): EVENT GAME_OVER ATTACKER
```

6.4. Diagrama de Secuencia de una Partida Completa

```
Atacante        Servidor         Defensor
   |               |                |
   |-- AUTH ------>|<----- AUTH ----|
   |<- OK/ROLE ----|---- OK/ROLE -->|
   |               |                |
   |-- CREATE_ROOM->|               |
   |<- ROOM_CREATED|                |
   |               |                |
   |-- JOIN 5 ---->|<----- JOIN 5 --|
   |<- OK ----------|----- OK ----->|
   |               |                |
   |-- START ------>|<----- START --|
   |<- GAME_STARTED |-- GAME_STARTED>|
   |                |-- RESOURCE_INFO>|
   |               |                |
   |-- MOVE/SCAN -->|               |
   |<- FOUND -------|               |
   |               |                |
   |-- ATTACK 0 -->|-- EVENT ATTACK>|
   |<- OK ---------|<--- DEFEND 0 --|
   |<- DEFENDED ----|---- OK ------>|
```

```
    |                |                |
    |       (o bien, timer expira)    |
    |<- ATTACK_TIMEOUT 0 ----------->|
    |<- GAME_OVER ATTACKER ---------->|
```

6.5. Prueba Manual con Telnet

Para verificar el protocolo sin necesidad de un cliente completo:

```
# Terminal 1: Iniciar el servicio de identidad stub
python3 server/identity_stub.py 9090

# Terminal 2: Iniciar el servidor de juego
export IDENTITY_HOST=localhost
export IDENTITY_PORT=9090
cd server && ./server 8080 server.log

# Terminal 3: Conectar como atacante
telnet localhost 8080

# Terminal 4: Conectar como defensor
telnet localhost 8080
```

7. Parámetros del Servidor

Parámetro	Fuente	Default	Descripción
Puerto de escucha	CLI: `./server <puerto>`	—	Puerto TCP del servidor
Archivo de logs	CLI: `./server <p><log>`	—	Ruta del archivo de logging
Host de identidad	Env: `IDENTITY_HOST`	localhost	Hostname del servicio de identidad
Puerto de identidad	Env: `IDENTITY_PORT`	9090	Puerto TCP del servicio de identidad
Timeout de inactividad	Env: `CGSP_IDLE_TIMEOUT`	600 s	Segundos sin actividad antes de cerrar la conexión
Tiempo máx. de partida	Env: `CGSP_GAME_TIMEOUT`	600 s	Duración máxima de una partida antes de declarar victoria defensora

Cuadro 9: Parámetros de configuración del servidor

8. Parámetros del Mapa de Juego

Parámetro	Valor
Dimensiones del mapa	100×100 unidades
Recursos críticos por sala	2 (posición aleatoria al crear sala)
Radio de detección (SCAN)	12 unidades (distancia euclidiana)
Tiempo límite de defensa	30 segundos
Tiempo máximo de partida	600 segundos (10 minutos); victoria del defensor al expirar
Máx. jugadores por sala	10
Máx. salas simultáneas	20

Cuadro 10: Parámetros del mapa del juego

9. Consideraciones de Seguridad

- Las contraseñas **no se almacenan** en el servidor de juego; se reenvían al servicio de identidad en cada autenticación.

- Las contraseñas **no se registran** en los logs del servidor (aparecen como `[***]`).
- Cada cliente opera en un hilo independiente; un cliente malicioso no puede bloquear a los demás.
- El timeout de inactividad de 600 s mitiga conexiones "zombie" que consuman hilos del servidor indefinidamente.

Advertencia de seguridad — Ausencia de cifrado. Esta versión del protocolo transmite todos los mensajes, incluyendo credenciales de autenticación, en **texto plano sin cifrado**. Esto representa un riesgo real de interceptación de contraseñas en redes no confiables.

La solución correcta para una versión de producción es envolver la capa TCP con **TLS 1.3** (usando, por ejemplo, OpenSSL o mbedTLS), lo que eliminaría este vector de ataque sin cambiar el formato de mensajes de CGSP. Esta mejora queda fuera del alcance del presente proyecto académico.

Fin del documento RFC-CGSP-2 — CyberGame Socket Protocol v2.0